Task 3 – Infrastructure as Code (IaC) with Terraform

Objective

The objective of this task was to learn and implement Infrastructure as Code (IaC) using Terraform. The goal was to define, provision, manage, and destroy infrastructure using declarative configuration files rather than manual setup.

Project Overview

For this task, I extended my existing Notes API project by adding a Terraform-based infrastructure example.

The implementation uses the Docker provider to provision infrastructure locally and demonstrate core Terraform concepts without requiring cloud resources.

The project demonstrates:

- Terraform Providers
- Terraform Resources
- Variables
- Outputs
- State Management
- Infrastructure Lifecycle Management

Technologies Used

- Terraform v1.15.5
- Docker Desktop
- Docker Provider
- Nginx
- Git & GitHub

Task Requirements

The following requirements were implemented:

- Terraform configuration files
- Provider configuration
- Resource creation
- Variables
- Outputs
- Terraform state management
- Infrastructure provisioning
- Infrastructure destruction
- Documentation

Project Structure

```
terraform/
├── main.tf
├── variables.tf
├── outputs.tf
└── README.md
```

Terraform Configuration

Provider

The Docker provider was configured to allow Terraform to interact with the local Docker daemon.

```
provider "docker" {}
```

Resources

The following resources were provisioned:

Docker Image

nginx:latest

Docker Container

terraform-nginx

Port Mapping

localhost:8081 -> container:80

Variables

Terraform variables were used to avoid hardcoded values.

Variable	Description
image_name	Docker image to deploy
container_name	Name of container
external_port	Host port exposed by container

Benefits:

- Reusability
- Configurability
- Maintainability

Outputs

Terraform outputs were configured to display useful information after deployment.

Outputs:

- container_name
- container_id
- container_url

Example:

container_name = "terraform-nginx"
container_url = "http://localhost:8081"

Terraform State

Terraform stores infrastructure information inside:

terraform.tfstate

Purpose:

- Tracks managed resources
- Detects infrastructure changes
- Enables updates
- Enables safe destruction of resources

State management is a core Terraform concept because it allows Terraform to compare the desired configuration with the actual infrastructure.

Infrastructure Architecture

Terraform
  |
  ▼
Docker Provider
  |
  ▼
Docker Image (nginx)
  |
  ▼
Docker Container
  |
  ▼
localhost:8081

Commands Used

Initialize Terraform

terraform init

Downloads providers and initializes the working directory.

Review Execution Plan

terraform plan

Displays planned infrastructure changes before deployment.

Create Infrastructure

terraform apply

Creates the Docker image and container.

View Outputs

terraform output

Displays information about provisioned resources.

Destroy Infrastructure

terraform destroy

Removes all infrastructure managed by Terraform.

Validation

The following validations were performed:

- Terraform initialized successfully
- Docker provider loaded successfully
- Execution plan generated successfully
- Docker image created successfully
- Docker container created successfully
- Outputs generated successfully
- Nginx accessible through browser
- Infrastructure destroyed successfully

Screenshots

Screenshot 1 – Terraform Plan

Description:

Shows Terraform identifying resources to be created.

Screenshot 2 – Terraform Apply

Description:

Shows successful infrastructure provisioning and generated outputs.

Screenshot 3 – Running Docker Container

Description:

Shows the Nginx container running after Terraform deployment.

Screenshot 4 – Terraform Destroy

Description:

Shows successful removal of all managed infrastructure resources.

Repository Information

GitHub Repository:

https://github.com/shantam-sharma/Notes-Api

Terraform Directory:

terraform/

Conclusion

Terraform was successfully used to provision, manage, and destroy infrastructure using Infrastructure as Code principles. The implementation demonstrates providers, resources, variables, outputs, state management, and lifecycle operations while maintaining reproducible infrastructure through declarative configuration.